

**Architecture
Design**

Hermes Profile Architecture

Math & Librarian Profiles

Mehdi Ghasemi

Project Proposal

September 2026

Executive Summary

This document records the complete design of the two Hermes agent profiles that power my mathematical research workflow — `math` and `librarian` — and specifies an abstraction layer under which either profile can be re-pointed at any future math project without modification. Sanitized, ready-to-use copies of both profiles ship with this document as the `math/` and `librarian/` folders (Section 4); the companion artifacts listed in the closing Sources and Verification note let every claim here be re-derived from a live installation.

The architecture resolves into five layers: a shared *Hermes core*, an isolated *profile system*, a *skill catalog* (capability-gated per profile), an *MCP bridge* (native servers plus a generic CLI-adapter pattern), and an *external service plane*. The math profile is a full-stack researcher with a sandboxed toolchain environment; the librarian profile is a capability-degraded derivative that shares every registration but disables the computational surface and adds ingestion skills of its own.

One structural finding drives the abstraction design: the librarian profile's MCP servers import scripts from the math profile's directory tree, so it cannot exist without the math profile. Section 10 proposes a capability/manifest model — persona templates, path interpolation, a shared adapter, a service registry, and per-project binding files — that removes every identified coupling in five concrete steps.

Contents

1 Objectives and Scope	3
2 Layered Architecture	3
3 Profile System: Structure Comparison	4
4 Folder Structure	5
5 Skill System: Design for General Math Projects	7
6 MCP Bridge	8
7 Tool Stack and Service Plane	9
8 The Research Workflow	11
9 Identified Couplings and Defects	12
10 Toward an Abstraction Layer	12
11 Roadmap for AgentDesign Itself	14
A The Self-Hosted Service Plane	15
A.1 Component inventory	15
A.2 Role in the six-stage workflow	17
A.3 Dependencies outside the stack	17
A.4 Why these tools	18
A.5 Deployment notes	18
A.6 Operating envelope: hardware, models and the offline posture	21
B The Model Context Protocol Bridge	21
B.1 Conventions of the bridge	22
B.2 What each server exposes	22
B.3 Notes on the bridge	24
C The Skill Surface	25
C.1 The designated workflow surface	25
C.2 Notes on the catalog	25

I Objectives and Scope

The `AgentDesign` project has three objectives:

1. **Document the as-is architecture.** Record, with file-level evidence, how the math and librarian profiles are composed: directory layout, configuration sections, skill catalogs, MCP server registrations, external services, scheduled jobs, and state stores. This is Sections 2–8.
2. **Identify the coupling points.** Enumerate every place where a profile depends on the specific current project layout — absolute paths, cross-profile imports, hardcoded endpoints, and stale references — so that the design of record distinguishes what is load-bearing from what is incidental (Section 9).
3. **Specify an abstraction layer.** Define how profiles become reusable personas: independent of any particular math project, instantiable for a new project with zero or one line of configuration, and testable against a capability contract (Section 10).

The scope is the two active research profiles. The remaining four registered profiles (`default`, `coder`, `advisor`, `trader`) are out of scope; their existence is noted only in Section 3. This project does not modify any live profile configuration — it is a documentation and design artifact.

2 Layered Architecture

Figure 1 gives the visual layout of the infrastructure under which both profiles operate. Five layers are distinguished; each lower layer is shared, each upper layer is isolated per profile at `~/.hermes/profiles/<name>/`.

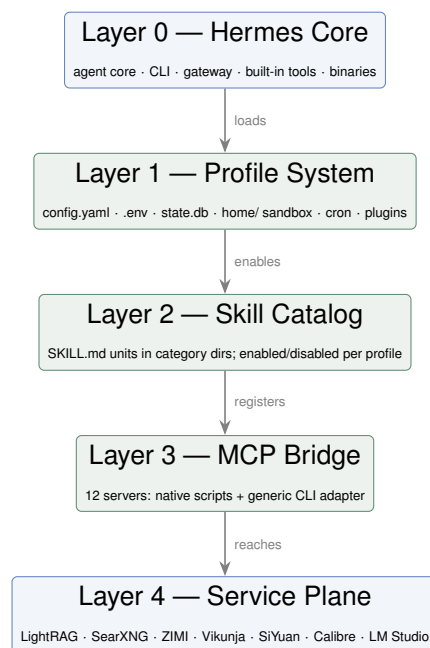


Figure 1: Five-layer profile architecture. Layers 2–3 are the capability surface of a profile; layers 0 and 4 are shared infrastructure. Arrows indicate the direction in which each layer is configured by, or depends on, the layer above it.

Layer 0 — Hermes Core

The core lives at `~/.hermes/hermes-agent/`: agent runtime, CLI (profile switching, tool registry), platform gateway (Telegram/WhatsApp/webhook adapters), built-in tools (terminal, browser, vision, file I/O, web search, kanban, memory) and the MCP discovery layer (`tools/mcp_tool_*.py`). Shared binaries (tirth security scanner, uv, Python 3.14, Node 26, ripgrep, Chromium) live in `~/.hermes/tools/`; all MCP stdio servers run from a single shared virtualenv at `~/.hermes/mcp-servers/.venv/`.

Layer 1 — Profile System

Each profile is an isolated directory with its own configuration, credentials, state and sandbox. The two profiles under study:

- **math** — full-stack mathematical researcher. It owns a sandboxed home environment carrying the local toolchain (Section 3), an active messaging gateway, and the scheduled literature jobs.
- **librarian** — literature manager (“manage and maintain books and articles”). It keeps an empty sandbox home, registers no scheduled jobs and no messaging platform, and reuses the same kanban assignment as math — itself a coupling noted in Section 9.

Both profiles run the same local model through LM Studio, with an `on_session_start` hook that guarantees the model server is up before each session. Long-term memory in both profiles is backed by Honcho (`memory.provider: honcho`).

Layer 2 — Skill Catalog

Skills are Markdown-defined capability units (`SKILL.md` with YAML frontmatter), organized in category directories, optionally carrying scripts and credential files. Each profile holds the same catalog — nine populated categories plus a handful of top-level skills, and the same empty category shells — and differs only in which entries are activated. The per-profile inventory extracted from a live installation is kept in `references/skill_inventory.md`; Section 5 summarizes the design-relevant parts.

Layer 3 — MCP Bridge

Skills reach services either directly (terminal/shell) or through Model Context Protocol servers registered in each profile’s config; every registered tool is exposed to the model under a name built from the server and the tool, joined by double underscores — for example, the vikunja task-creation tool. Section 6 documents the two server families and the cross-profile dependency this layer creates.

Layer 4 — Service Plane

A fixed set of self-hosted services on the local network, plus SaaS APIs and a local inference server. Table 4 lists them with their role in the workflow.

3 Profile System: Structure Comparison

Table 1 compares the two profiles as designs rather than as installations: the question is what each profile *is*, not what it happens to contain today.

Table 1: Profile design comparison: the librarian is a capability-degraded derivative of the math profile, not an independent design.

Design element	math	librarian
Role	Full-stack mathematical researcher	Literature manager: “manage and maintain books and articles”
Inference	Local model served by LM Studio	Same model and provider, with an explicit API mode
Skill surface	Full catalog; computation, formal proof and manuscript skills active	Same catalog with the computational surface disabled in configuration; adds its own ingestion skills (<code>calibre-library-ingestion</code> , <code>djvu-to-pdf</code>)
MCP bridge	Native servers plus adapter-wrapped CLI tools	Identical registrations, all resolving into the math profile’s tree
Plugins	<code>vikunja</code> and <code>wiki-browser</code> enabled	Identical
Toolchain sandbox	Lean 4, Rust, SageMath and TeX Live reachable from the profile home	Empty stub
Scheduled jobs	Literature scan and wiki maintenance	None
Messaging platform	Active gateway	None
Cross-session memory	Honcho	Honcho
Kanban assignment	math	math — inherited, not chosen

The librarian is a capability-degraded derivative of math, not an independent design: it inherits every skill directory and MCP registration, then suppresses the computational surface via the `skills.disabled` list. It also carries two skills that exist only in its own tree — `calibre-library-ingestion` and `djvu-to-pdf` (productivity category) — which is where its genuine specialization lives.

The math profile carries a sandboxed home directory that stands in for a machine-local toolchain. It holds a Lean 4 installation managed by `elan`, a Rust toolchain together with its registry and download caches, a dedicated browser profile for NotebookLM grounded synthesis, runtime state directories for SageMath and Maxima, and the font and format caches of TeX Live. Two properties matter for the abstraction design. First, most of the space is disposable package cache rather than toolchain state: the load-bearing parts are the Lean and Rust installations, everything else can be re-provisioned. Second, the sandbox is not self-contained — the SageMath and TeX Live binaries resolve to host-level installations rather than to anything inside the profile home — so a profile’s computational capability depends on the host even when it looks self-contained.

4 Folder Structure

Three directory levels carry the design: the shared Hermes root, the two profile trees, and the per-project binding file. The listings below show the structure with the design-relevant entries; the profile skeleton in Listing 4 marks which entries exist in which profile.

Shared root

The `~/hermes/` root holds everything that is not profile-scoped. Two entries are load-bearing for the coupling analysis: `mcp-servers/` holds the single Python environment that every MCP server of every profile runs in,

and `skills/` is the shared pool that both profiles fall back to for both skills and plugins.

```

1 ~/.hermes/                # shared Hermes root
2 config.yaml               # global defaults; profiles override
3 profiles/                 # one directory per profile (math, librarian, advisor, ...)
4 skills/                   # shared skill pool: 17 categories
5 plugins/                  # shared plugin installs: vikunja, wiki-browser, hermes-
   achievements
6 desktop-plugins/          # desktop plugin surfaces: vikunja, wiki-browser
7 plugin-data/              # per-plugin persistent state: hermes-achievements
8 mcp-servers/.venv/        # ONE Python env for all 13 MCP servers
9 memories/ sessions/ state.db # memory files, session store, global state
10 kanban/ kanban.db         # kanban board + dispatcher state
11 cron/ hooks/ agent-hooks/ # scheduling and lifecycle hooks
12 bin/ node/ webui/ tools/  # runtime binaries, desktop build, CLI tools

```

Profile tree: the two profiles side by side

The math profile is a structural superset: it carries the runtime residue of an active profile, while the librarian keeps only what its role needs. Two differences are design-relevant — `scripts/`, the MCP adapter directory, exists only on the math side, and `home/` is populated only there. Those two facts are the structural root of the path coupling in Section 6.

1	profiles/math/		profiles/librarian/
2	config.yaml	profile.yaml	config.yaml profile.yaml
3	.env		.env
4	skills/	24 top-level dirs	skills/ 23 top-level dirs
5	plugins/	vikunja, wiki-browser,	plugins/ empty (borrows shared pool)
6		hermes-android (disabled)	
7	desktop-plugins/	vikunja, wiki-browser	desktop-plugins/ [absent]
8	scripts/	mcp_cli_adapter.py + 8 .py	scripts/ [absent]
9	home/	toolchain sandbox	home/ (empty stub)
10	scratchpad/	failure scratchpad	scratchpad/ [absent]
11	cron/ hooks/ memories/ sandboxes/		cron/ hooks/ memories/ sandboxes/
12	sessions/ workspace/ cache/ logs/		sessions/ workspace/ cache/ logs/
13	backups/ state.db SOUL.md		backups/ state.db SOUL.md
14	telemetry/	[absent]	telemetry/ (sender on)
15	vault/	[absent]	vault/ (credential store)
16	state.db / SOUL.md / backups/		state.db / SOUL.md / backups/

Beyond the entries shown, the librarian keeps `telemetry/` and `vault/`; math's extra entries are dominated by runtime residue (gateway sockets, backup snapshots, retired write-ahead logs) plus the three design-relevant directories listed above.

Skill tree layout

Both profiles carry the same directory layout — the same populated categories, the same reserved shells, the same four top-level skills; activation, not layout, is what differs.

```

1 profiles/<profile>/skills/ # 22 top-level dirs, identical in both
2 research/ productivity/ mlops/ # 9 populated categories; each holds
3 software-development/ github/ devops/ # <skill>/SKILL.md with optional
4 web/ autonomous-ai-agents/ social-media/ # scripts/ and references/
5 apple/ creative/ data-science/ email/ # 9 empty category shells (reserved,
6 environment/ media/ note-taking/ # no SKILL.md; used by the curator

```

```

7  mathematical-research/ smart-home/          # as retirement targets)
8  user-latex-style/ hermes-plugin-management/  # 4 top-level skills , present in
9  semantic-scholar/ workspace-doc-hierarchy/  # both profiles
10 .archive/                                     # retired skills kept as curator tombstones
11 .curator_ledger.jsonl .curator_state/ .curator_suppressed/
12 .bundled_manifest .usage.json .hub/ .locks/
13                                     # ^ per-installation state: not shipped

```

Distributable profile packs

The `math/` and `librarian/` folders inside this project are sanitized, ready-to-use copies of the two profiles: the same directory structure, the same configuration and skill trees, with every credential, endpoint, hostname and local path replaced by a placeholder, and with the per-installation bookkeeping (the loader, curator and telemetry files listed above) left out. A builder script in `references/` copies the configuration, skill, plugin, script and memory trees from a live installation, scrubs secrets and identity, and writes a per-pack `README.md`. Copying a pack into `~/.hermes/profiles/` and filling in the placeholders reproduces the profile; the packs are described in Section 4.

This document's own folder

```

1 AgentDesign/
2  agent_design.tex          # this document (source of record)
3  agent_design.pdf         # compiled output
4  AGENTS.md               # project guidance for future sessions
5  math/ librarian/        # sanitized , ready-to-use profile packs (Section~\ref{sec:
   packs})
6  stack/                  # the profiles' service plane as one compose project
7  cpistuff/               # CPI LaTeX template: cpi.sty + logos
8  t1ggm.fd ts1ggm.fd      # Garamond T1 declarations (compile prerequisite)
9  data/profile_comparison.json # machine-readable extraction (secret values redacted)
10 references/extract_profile_data.py references/build_profile_packs.py
11 references/sanitize_extracted_data.py
12 references/skill_inventory.md # per-profile skill inventory
13 .gitignore

```

The `stack/` folder turns the service plane of Section 7 into an executable artifact: one compose project that brings up every container the two profiles talk to, with credentials interpolated from an untracked `.env`. Its `README.md` maps each container to the MCP server or skill that consumes it and records what the live deployment gets wrong.

Reproducing the build requires one environment step beyond the template: the CPI style loads the Garamond T1 fonts, which are not part of a stock TeX Live installation. They are installed per-user (no root) under `~/texmf/` — `ggm*.tfm` in `fonts/tfm/ggm/`, `ggm*.vf` in `fonts/vf/ggm/`, `ggm*.pfb` in `fonts/type1/ggm/`, `ggm.map` in `dvips/ggm/`, then `mktexlsr ~/texmf`. Without it the build fails at the first page with a missing-font error rather than a document error.

5 Skill System: Design for General Math Projects

Skills are the unit of reusability. Each `SKILL.md` carries a trigger description in its frontmatter, an imperative body, optional scripts, and — where needed — a credential file (`.env`) co-located with the skill that consumes it. The catalog is shared between the two profiles: the same populated categories, the same empty category

shells, and the same top-level skills. Only activation differs. Table 2 summarizes the catalog by design role; the per-entry inventory extracted from a live installation is in `references/skill_inventory.md`.

Table 2: Skill catalog by category, with activation per profile.

Category	Design role	Activation
research	The research surface proper: the six-stage workflow, symbolic and numeric computation, formal verification, literature retrieval and the wiki pipeline	full in math; computational and proof skills disabled in librarian
productivity	Knowledge management: task ledger, notes, bibliography, manuscript production, document conversion	full in both; the librarian adds its ingestion skills
mlops	Local inference operations and pipeline tooling	reduced in librarian
software-development	Hermes self-extensibility, planning, subagent orchestration, desktop inspection	full in math; partly disabled in librarian
github, devops, agents, web	Repository operations, SDLC review, delegation, blocked-page recovery	mixed; review and delegation off in librarian
top-level skills	Cross-cutting capabilities: manuscript style, plugin management, citation graph, document hierarchy	full in math; the librarian disables the development-oriented entry

Three observations are design-relevant:

1. **The math research surface is exactly the set of skills a generic math project needs:** decompose $\rightarrow$ literature $\rightarrow$ compute $\rightarrow$ verify $\rightarrow$ write, implemented by `math-research-workflow`, whose frontmatter names the sibling skills it relies on. This set, not the full catalog, defines the reusable “mathematician” capability contract (Section 10).
2. **The librarian’s disabled list is a negative specification of that same contract:** every computational skill (`sagemath-mcp`, `sympy-mcp`, `lean4`, `lean4-goedel-agent`, `scientific-coding`) are turned off. Capability gating by negation works today but is brittle in a subtler way than it looks: entries resolve by *leaf name* against the profile’s skill tree, which includes the curator tombstone directory `skills/.archive/`. Every disabled name therefore resolves to a real `SKILL.md`; none dangles. But a sizeable minority of them resolve *only* under `.archive/` — they are residue from skills the curator has already retired (`docx`, `opencode`, `test-driven-development`). Because resolution is name-based, any of these stale entries would silently disable a same-named skill if it were ever reinstalled. One inversion deserves note: `reddit-reading` is disabled for math but enabled for librarian — the only skill whose gate runs opposite to every other difference between the two profiles.

6 MCP Bridge

Twelve MCP servers are registered per profile, all launched from one shared interpreter environment. They fall into three families:

Table 3: MCP server registrations. “Pattern” distinguishes native servers from generic CLI-adapter wraps; target paths are written relative to the profile tree.

Server	Pattern	Target	Tools
sagemath	native	skills/research/sagemath-mcp/	10
lean4	native	skills/research/lean4/	4
searxng	native	skills/research/searxng/scripts/	1
vikunja	CLI adapter	skills/productivity/vikunja/	15
wolfram-alpha	CLI adapter	skills/research/wolfram-alpha/	3
lightrag-query	CLI adapter	skills/research/lightrag/	3
siyuan	CLI adapter	skills/productivity/siyuan/	7
zimi	CLI adapter	skills/research/zimi/	4
zotero	CLI adapter	skills/productivity/zotero/	7
scientific-coding	CLI adapter	skills/research/scientific-coding/ (CPU and memory limits via env)	4
latex-manuscript	CLI adapter	skills/productivity/latex-manuscript/	5
calibre	native, librarian only	math: environment block without a command; librarian: a server with its own virtual environment	12

The **generic CLI-adapter pattern** is the load-bearing design of this layer. A single script — the generic adapter kept in the profile’s `scripts/` directory — wraps any argparse-based Python CLI tool as a stdio MCP server: it introspects the target’s parser at startup, exposes each subcommand as an MCP tool with JSON-schema arguments, and reconstructs command lines from tool calls. Most of the registered servers use it; adding a new service-backed skill therefore requires only a thin CLI script plus a config entry (endpoint and token via environment), with no protocol code.

Two cross-profile facts follow directly from the table:

- Every path in every MCP registration — including all of the librarian’s — points into the **math profile’s directory tree**: every adapter tool script, every native server script, and the adapter itself. The librarian has no `scripts/` directory of its own. Deleting or moving the math profile would silently break every tool-bridged capability of the librarian.

7 Tool Stack and Service Plane

Table 4 lists the services both profiles reach, grouped by role in the research workflow. Endpoints and credentials are not part of the design: each profile resolves them from its own environment file at load time, so the same registrations serve any deployment. The profiles share the bulk of their environment keys; the math-only keys are messaging and auxiliary settings (Telegram bot token and allowlist, WhatsApp mode fields, an OpenRouter key).

Table 4: Service plane, grouped by role in the research workflow. Both profiles share the same registrations; differences are noted.

Service	Role in the workflow	Integration
LightRAG	Graph-RAG over the curated literature lake: ingestion and multi-hop query	MCP
SearXNG	Federated web metasearch with structured output	MCP
ZIMI	Offline encyclopedia for archival grounding before any live search	MCP
Vikunja	Task ledger: hypothesis trees, phase tracking, cross-session state	MCP and plugin
SiYuan	Block-level notes: human-readable findings and equation blocks	MCP and plugin
Calibre	Ebook library management and ingestion	librarian only
Zotero	Bibliography, BibTeX export, citation verification	MCP
Wolfram Alpha	Symbolic and numeric computation beyond the local stack	MCP
Honcho	Persistent cross-session research memory	built-in memory provider
LM Studio	Local inference for both profiles	model provider
Portainer	Container management of the service plane itself	terminal
Semantic Scholar, arXiv, Crossref	Citation-graph layer and the metadata verification gate	skills

The **four integration channels** a skill may use, in the preference order codified by `math-research-workflow`:

1. *MCP native / adapter tools* — fastest path into services; no shell involved.
2. *Terminal invocations of CLI scripts* bundled with skills (e.g. the arXiv research script, wiki ingestion pipeline).
3. *Built-in Hermes tools* — web search/extraction, file I/O, browser automation.
4. *Local binaries from the sandbox home* (SageMath, Lean 4 via `elan/rustup`, TeXLive) for computation and compilation that must stay on-machine.

Plugins

Plugins are the third extension channel beside skills and MCP servers, and their configuration is byte-identical in the two profiles.

Two facts matter for the abstraction design. First, the librarian enables both plugins while its own `plugins/` directory is empty: plugin resolution falls back to the shared pool exactly as skill resolution does, so the librarian's plugin capability is entirely borrowed — and, as with skills, the coupling is by name rather than by manifest. Second, `hermes-android` sits in `math's plugins/` directory but is disabled, while the plugin that carries credentials (`vikunja`, whose dashboard ships a co-located `.env` file) is enabled in both profiles: plugin credentials, like skill credentials, live inside the tree that ships them, not in a profile-level secret store.

Table 5: Plugin configuration.

Element	State
Enabled plugins	vikunja, wiki-browser — identical in both profiles
Disabled plugins	hermes-android — identical in both profiles
Tool-override entries	hermes-android and wiki-browser, both with overrides denied
Profile plugin directory	math carries the plugin trees; the librarian's is empty
Shared installs	resolved from the shared Hermes plugin pool when a profile has no local copy
Desktop surfaces	desktop-plugins/ carries the vikunja and wiki-browser surfaces

8 The Research Workflow

The designated workflow for math projects is the `math-research-workflow` skill (v1.6), a single-agent adaptation of the MathAgent multi-agent system with Graph-of-Agents message passing and Reflexion quality gates. Its six stages, each gated by a 5-check Reflexion gate:

Table 6: The six stages of the designated research workflow, each gated by a five-check Reflexion gate.

Stage	Activity	Primary tools
1. Problem decomposition	Restate the problem; open Vikunja tasks and a hypothesis tree (conjecture = parent, attack branches = children with status)	vikunja MCP
2. Literature synthesis	Local-first retrieval cascade: LightRAG → academic-research-hub → Semantic Scholar → SearXNG → ZIMI; every citation passes the <i>Citation Verification Gate</i> (arXiv API / Crossref / S2 before any bibliographic fact is recorded)	lightrag, searxng, zimi, s2 skill
3. Computation	Deterministic-first: SymPy → SageMath → Wolfram Alpha → sandboxed scientific-coding; Error-Feedback Loop with a 3-retry budget, exact prior error carried into each retry	sympy-mcp, sagemath-mcp, wolfram-alpha, scientific-coding
4. Formal verification	Three-stage Lean pipeline: formalize → compile clean (<code>lean4_check</code> against an explicit Mathlib project) → statement-fidelity diff vs. the informal claim; only then is a theorem ledger-marked “proven & Lean-verified”	lean4, lean4-goedel-agent
5. Manuscript production	Drafter → Enhancer → Reviewer pass over LaTeX with the user-latex-style anti-pattern checklist and notation audit	latex-manuscript MCP, user-latex-style
6. Reflection & close-out	Claims ledger updated; failures promoted to skill pitfalls or project reports; project README + AGENTS.md status synced in the same commit	—

Supporting state: a per-project failure scratchpad under `~/hermes/profiles/math/scratchpad/`, per-project verified-source registries for citation dedup, and the cron jobs (weekly arXiv scan in the five-topic overlap set; bi-weekly wiki maintenance) that feed Stage 2 continuously. The shared numerical backend for polynomial-optimization experiments — moment and SOS hierarchies, SONC relaxations — is invoked from its own environment by shell rather than through the bridge; it is a project dependency, not part of the profile design, and is therefore outside the scope of this document.

9 Identified Couplings and Defects

Ranked by severity, with the evidence for each entry given in the consequence column:

Table 7: Coupling points that bind the profiles to the current project layout.

Coupling	Consequence	Severity
Cross-profile script import (librarian → math)	Every adapter tool script, every native server script and the adapter itself resolve under the math profile; the librarian has no scripts directory of its own	critical
Absolute skill paths in MCP configuration	Every registration hardcodes the full home path; no interpolation mechanism exists	high
Hardcoded service endpoints	Environment files embed host and port directly; no name-based resolution	medium
Project-pinned working directories	Working directory, repository scan roots and execution mode reference specific project trees; the kanban orchestrator is fixed to one profile in both configurations	medium
Toolchain baked into the profile home	The local computation toolchain ships as profile state rather than as a project binding	medium
Stale disabled entries (curator tombstones)	Disabled names that resolve only under the curator archive would silently disable a same-named skill if it were reinstalled	low
Asymmetric service registration	The same service name means different things in the two profiles	low

10 Toward an Abstraction Layer

The goal is a design in which *profiles are personas and projects are bindings*. A profile declares **what it can do** (capabilities) and *how it behaves* (persona); a project declares **where things live** (paths, endpoints, ledgers). Nothing in today's configuration should need editing to attach the existing math or librarian profile to a new math project.

A. Capability-gated skill activation (replaces negative lists)

Each `SKILL.md` gains one frontmatter field:

```
requires: [computation, formal-proof] # empty list = always available
```

and each profile declares a positive capability set instead of a disabled list:

Persona	Capabilities
mathematician (today: math)	literature, computation, formal-proof, manuscript, knowledge-mgmt, optimization-backends
librarian	literature, ingestion, knowledge-mgmt, manuscripts-light

The loader activates a skill iff `requires` $\subseteq$ `persona.capabilities`. The librarian's current negative list disappears; tombstone names (entries that resolve only under `.archive/`) become impossible to carry forward, since a capability gate cannot name a skill the persona does not declare; and a new persona is one YAML block, not a diff of every registration.

B. Path interpolation and a shared adapter (removes the critical coupling)

Introduce `${PROFILE_HOME}` and `${HERMES_ROOT}` tokens in `mcp_servers` entries, mirroring the `{baseDir}` mechanism skills already use. Concretely:

1. Move the generic adapter to a shared location: `~/.hermes/tools/mcp_cli_adapter.py` (next to `tirith/uv`, which are already cross-profile). Adapter-wrapped tool scripts move with their skills — they do, since `TOOL_SCRIPT` paths become `${PROFILE_HOME}/skills/...`
2. Each profile's config then references only its own tree; the librarian stops importing from math. Verification is trivial: no path in a profile's config may contain another profile's name.

C. Service registry (removes the endpoint coupling)

A `services.yaml` naming services abstractly, with endpoints resolved from env at load time:

```
1 # ~/.hermes/services.yaml
2 lightrag: { url: ${LIGHTRAG_URL}, key: ${LIGHTRAG_API_KEY} }
3 searxng: { url: ${SEARXNG_URL} }
4 zimi: { url: ${ZIMI_URL} }
5 vikunja: { url: ${VIKUNJA_URL}, token: ${VIKUNJA_TOKEN} }
6 siyuan: { url: ${SIYUAN_URL}, token: ${SIYUAN_TOKEN} }
7 calibre: { url: ${CALIBRE_BASE_URL} }
8 honcho: { url: ${HONCHO_BASE_URL} }
```

MCP entries then reference `$services.lightrag.url` etc. The `.env` files remain the secret store (never committed), but IPs move out of configuration logic into environment, so a service migration is an env edit, not a per-profile config surgery. This also fixes the calibre asymmetry: one definition, activated by capability ingestion.

D. Toolchain bundle (removes the sandbox coupling)

The sandbox home is a *capability artifact* (computation, formal proof and manuscript), not profile state. Split it: keep identity files (`.gitconfig`, `.ssh`, `.bashrc`) in the profile; extract the toolchain into an installable bundle `math-toolchain.tar` or a Docker/ conda-spec manifest that any persona declaring the relevant capabilities can provision on demand — and that a project without formal-proof needs (e.g. pure numerical work) never pays for.

E. Project binding files (the actual project-independence mechanism)

A new math project is then attached by a single file at its root:

```
1 # ~/Code/<NewProject>/.agent.yaml
2 profiles: [math, librarian]
3 working_dir: .
4 vikunja_project: <id> # task ledger target
5 lightRAG_collection: <project> # literature namespace
6 sage_env: sage # toolchain bundle component (conda env name)
```

Profiles discover projects by scanning their `repo_scan_roots`; a profile picks up a project's binding at session start and overrides `terminal.cwd`, ledger targets, and collection names from it. Nothing inside the profiles is edited when the project changes — which is exactly the property today's coupling list (Table 7) shows to be absent.

F. Remediation prerequisites

Before any of A–E is built, two live defects should be fixed in place (each a one-line or one-file change): prune the tombstone-only names from the disabled lists, and give math a real calibre server command or delete its env-only block. These are recorded here as open work items; none is applied by this documentation project.

Instantiation model, end state

Figure 2 summarizes the target relationship between templates, personas and projects.

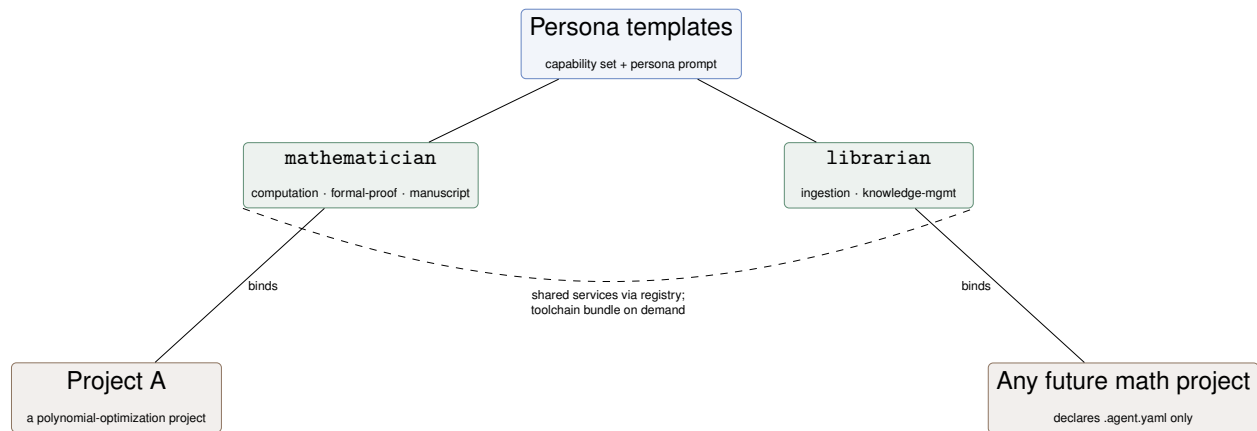


Figure 2: Target model: personas are instantiated from templates with a capability set; projects bind to personas through root-level binding files. No profile-internal configuration changes when the project changes.

1.1 Roadmap for AgentDesign Itself

Phase	Activity	Status
I. Exploration	Read every configuration, skill tree, state database and script; extract the design data programmatically	done
II. Documentation and packs	This document, the sanitized profile packs of Section 4, the skill inventory and the extracted data	this deliverable
III. Remediation PRs	The four one-line fixes of Section 10(F), as separate commits against the live profiles	open
IV. Abstraction pilot	Implement capability frontmatter + loader gate for one skill pair (e.g. <code>sagemath-mcp</code> under a test persona); verify librarian behavior is unchanged	open
V. Migration	Shared adapter move, <code>services.yaml</code> , toolchain bundle; then retire disabled lists	open

A The Self-Hosted Service Plane

Section 7 described the service plane as a design; this appendix records it as a deployment. The `stack/` folder of this project holds one compose project that brings up every container the two profiles talk to. On the live installation those containers run as seven separate compose projects under a single directory, one per subsystem, with credentials written directly into their files; the subject here is the unified version, in which the same sixteen components form one stack whose secrets live in an untracked environment file.

Sixteen components is more than the profiles' service table suggests, because four of them are infrastructure for another component rather than services the profiles call directly: the metasearch limiter store, the graph and vector stores behind the retrieval service, and the database and cache behind the memory service. They are listed here because a stack that omits them is not reproducible. The two inventory tables below therefore divide the components by function, and the tables that follow place them in the workflow.

A.1 Component inventory

Table 8: Component inventory, part one: the edge and the knowledge-management components.

Component	Endpoint	Image	Role in the workflow
npm	:80, :443, :81	jc21/nginx-proxy-manager:latest	Edge of the plane: terminates HTTP/HTTPS on the host, with the routing admin UI on the third port. No proxy hosts are configured yet, so every other component is reached by its own port.
portainer	:9000	portainer/portainer-ce:latest	Management of the plane itself: start, stop, inspect and read the logs of the other containers, driven from the terminal with an API key rather than through a tool bridge.
siyuan	:6806	b3log/siyuan:latest	Block-level note store: findings, synthesis pages and equation blocks, written through the notes tool bridge and the desktop plugin and read back when grounding an answer or drafting a manuscript.
vikunja	:3456	vikunja/vikunja:2.3.0	Task ledger: multi-day work, hypothesis trees (conjecture as parent, attack branches as children carrying a status) and cross-session state.
searxng	:5050	searxng/searxng:latest	Federated metasearch: the live-web layer of the retrieval cascade, returning structured results to the literature stage.
searxng-redis	internal	redis:7-alpine	Limiter store for the metasearch instance: engine bans, suspension timers and request accounting. Not published to the host.

Table 9: Component inventory, part two: retrieval, ingestion and research memory.

Component	Endpoint	Image	Role in the workflow
zimi	:8899	epheterson/zimi	Offline encyclopedia over ZIM archives: sub-second archival grounding for historical and encyclopedic claims, consulted before any rate-limited live search.
lightrag	:9621	ghcr.io/hkuds/ lightrag:latest	Graph-RAG over the curated literature lake: ingests documents and answers multi-hop entity and relation queries. Sole backend of the retrieval tool bridge.
neo4j	:7474, :7687	neo4j:5.20.0	Property-graph store holding the entity graph of the literature lake (browser and Bolt endpoints).
qdrant	:6333, :6334	qdrant/qdrant: latest	Vector store holding the chunk embeddings of the literature lake (HTTP and gRPC endpoints).
paperflow	:8000	built from ./ paperflow	Paper conversion: PDF to Markdown with OCR, feeding the ingestion skills. Jobs are queued and their outputs land in the shared data directory.
pocketbase	:8090	ghcr.io/ muchobien/ pocketbase:latest	Intake queue for the wiki pipeline: a document is dropped here and the pipeline carries it into the knowledge graph and the wiki, decoupling capture from processing.
honcho-api	:8008	built from ./ honcho	Persistent cross-session research memory: the profiles' memory provider, storing what was concluded and serving it back at the start of later sessions.
honcho-deriver	internal	built from ./ honcho	Background worker of the memory service: derives the peer representation and session summaries that the API serves.
honcho-database	127.0.0.1:5433	pgvector/ pgvector:pg15	Store for the memory service: messages, conclusions and their embeddings. Published on the loopback interface only.
honcho-redis	127.0.0.1:6379	redis:8.2	Cache and work queue for the memory service. Published on the loopback interface only.

A.2 Role in the six-stage workflow

Table 10: Where the stack sits in the six-stage workflow of Section 8.

Stage	Components	What they contribute
1. Problem decomposition	vikunja, siyuan	The restated problem becomes a ledger task with a hypothesis tree; notes hold the framing that a ledger entry is too terse to carry.
2. Literature synthesis	zimi, searxng, lightrag, neo4j, qdrant, pocketbase, paperflow	The retrieval cascade runs offline-first: the archive answers what it can, federated search covers the live web, and the graph-RAG service answers multi-hop questions over what has already been ingested. The ingestion components keep that lake current.
3. Computation	none of the stack	Deterministic computation runs against the sandboxed toolchain and the symbolic services; the stack supplies no part of this stage's critical path.
4. Formal verification	none of the stack	The proof assistant runs from the sandbox home against a pinned library project; verification is deliberately local and offline.
5. Manuscript production	siyuan	Manuscript sources are compiled locally; the note store carries the notation decisions and equation blocks that the drafting passes reuse.
6. Reflection and close-out	vikunja, honcho-api	Tasks are closed in the ledger and the outcome is written to the memory service, so the next session starts from what was decided rather than from what was merely said.
Every stage	npm, portainer, honcho-api	Edge access to the plane, container management while working, and the memory that spans sessions.

A.3 Dependencies outside the stack

Table 11: Dependencies the stack does not containerise, and why.

Dependency	Where it lives	Why it is out of scope
Inference server	another machine on the LAN, port 1234	Serves every model call for both profiles and for the memory service. Not containerised because it needs a GPU and multi-gigabyte model weights; both profiles fail loudly when it is unreachable.
Calibre server	systemd service on the host, port 9876	The ebook library and its conversion tooling. The library pre-dates the container stack, and the ingestion skills call it over HTTP.
Zotero	SaaS API	Bibliography, BibTeX export and the citation library.
Wolfram Alpha	SaaS API	Symbolic and numeric computation beyond the local stack, used in the computation stage.
Semantic Scholar, arXiv, Crossref	SaaS APIs	Citation graph and the metadata verification gate that every recorded citation must pass.
OpenRouter	SaaS API	LLM and embedding provider behind the graph-RAG service.

The first two rows are reached over the network exactly as the containers are; the rest are public APIs. Their absence from the stack is a design decision rather than an oversight: each needs hardware, data or a service that a container cannot supply, and each is reachable from the profiles on the same terms as the containerised components.

A.4 Why these tools

The components above were not assembled from a preference list; they are the ones that satisfied six constraints at once, and each constraint is a property the workflow depends on.

- **Self-hosted and open source.** The retrieval cascade is deliberately local-first, and a tier that bills per query or throttles by contract cannot be its first stop. Every knowledge and retrieval component runs on the same host as the profiles.
- **An HTTP API, not only a user interface.** A service becomes a tool only through the adapter pattern of Appendix B, which introspects a command-line tool and publishes it. A component whose only interface is a web page cannot be reached by the workflow at all.
- **One container, with its state in a volume.** The whole plane is a single compose project. Anything requiring a cluster, an operator or a managed control plane was out of scope before its features could be considered.
- **Storage in an open, inspectable format.** Markdown, Postgres, ZIM archives and a property graph are all readable without the tool that wrote them. The knowledge base is meant to outlive the tooling around it.
- **A footprint that fits a workstation.** The plane runs beside the research machine rather than in a data centre; the two components that need real resources — the inference server and the ebook library — were left outside the stack for that reason (Table 11).
- **A role with no cheaper local substitute.** Each component answers a call the six stages actually make: a task to open, a document to ingest, an article to ground, a formula to find again.

Two tables follow. The first covers the components the workflow writes to and reads from; the second covers retrieval and the platform beneath it. In both, the alternatives are live substitutes rather than rejected bids — nothing in the workflow depends on a product identity, because every service is reached through an HTTP endpoint and a thin adapter.

Two of the choices were not free, and the distinction is worth keeping. Cross-session memory is a capability the runtime provides through a provider interface, so replacing the memory service would be an integration project rather than a compose edit. And the graph-RAG service is the only component on the bridge that can answer a multi-hop question, so it is not interchangeable with the search tiers above it — the archive and the metasearch instance can each be swapped without touching the workflow, and this one cannot.

The converse also holds, and it is where the plane is thinnest. The edge component is provisioned but carries no routing, and the management component duplicates what the compose file already does. The two components whose absence would break a path are neither of them the edge: without the conversion service the ingestion path stalls, and without the memory service the continuity between sessions degrades to whatever a single context window can hold.

A.5 Deployment notes

The stack is a single compose project, and no credential is stored in it: every one is interpolated from an untracked environment file whose template ships with the project as a list of placeholders. Bind-mounted

Table 12: Why the knowledge and workflow services fit, and what would replace them.

Component	Why it fits	Alternatives, and the trade-off
SiYuan	Block-level API with first-class equation blocks. A finding can be written, searched and read back as a block, and a formula survives as a formula — which is what lets a verified expression travel from the computation stage into a draft.	Obsidian and Logseq are better personal editors and files-first, but neither exposes a server-side block API; Joplin stores notes but has a weaker block model; Notion would move the knowledge base off the machine.
Vikunja	Projects, tasks, statuses, due dates and comments behind a REST API broad enough to give the bridge its largest tool surface. A hypothesis tree is exactly a parent task with child branches carrying a status.	Kanboard is mature and simpler to run, with a thinner status model; Focalboard is board-centric; Nextcloud Tasks only makes sense where Nextcloud is already run; Linear and Todoist are hosted services.
PocketBase	A drop point with an API, which decouples capture from processing: a document is filed and the pipeline drains it later. It is the reason ingestion can be asynchronous.	n8n is far more capable and far heavier; Airflow is a batch scheduler, oversized for a queue; Huginn is agent-oriented with no file queue; a watched directory has no metadata and no status.
Honcho (api, driver, pgvector, redis)	Memory as a service the runtime already speaks, with a background worker that maintains the peer representation between sessions. What was concluded, and the session summaries, live in pgvector rather than in a prompt.	Mem0, Zep and Letta are comparable memory services, but each would need its own provider integration written before it could replace this one; the built-in markdown memory files the profiles also keep are durable but unstructured, and are no substitute for derived summaries.
PaperFlow	PDF to Markdown with OCR, queued, which is precisely the ingestion step the wiki pipeline needs; the pipeline behind it handles scanned books as well as digital ones.	marker, MinerU and Docling are all strong and none is packaged as a queue-backed service; GROBID is excellent at scholarly structure and references but narrower on general books; Mathpix gives the best mathematical OCR and is a paid API.

Table 13: Why the retrieval and platform components fit, and what would replace them.

Component	Why it fits	Alternatives, and the trade-off
SearXNG (with redis)	Federated metasearch with no API key and structured output: the live-web tier of the cascade without a SaaS contract or a per-query cost. The Redis companion is what keeps the instance from being banned upstream.	Whoogle fronts a single engine; 4get is similar in spirit; Brave Search and SerpAPI are paid external services that reintroduce exactly the contract this tier exists to avoid.
ZIMI	Serves ZIM archives — the offline-encyclopedia format — over HTTP with search and clean-text endpoints, which is what makes sub-second archival grounding possible before any rate-limited search. A query can be aimed at one archive, such as a mathematics subset.	Kiwix-serve, the canonical ZIM server, reads the same archives with a plainer API; reading the archives from disk without a service is possible but loses search and text extraction.
LightRAG (with neo4j and qdrant)	A graph-RAG service with a REST API: entities and relations over the literature lake, answering multi-hop questions the vector tier alone cannot. The Neo4j and Qdrant containers are its stores, so one service covers graph and vectors.	Microsoft GraphRAG builds richer community summaries but costs far more to index and is not a service; RAGFlow is document-centric; a hand-built LlamaIndex and Neo4j pipeline gives full control and all of the maintenance; plain vector search removes two components and the multi-hop answers with them.
Nginx Proxy Manager	One reverse proxy for the whole plane, with a UI for certificate issuance. With one host and many ports, TLS termination is the only thing the edge is asked to do.	Traefik is declarative and better when the routing lives in code; Caddy has the simplest automatic HTTPS and no UI; plain nginx gives full control with manual certificates. On the live installation no proxy hosts are configured yet, so this component is provisioned and idle.
Portainer	Inspect, restart and read the logs of the plane from a terminal API, without spending a bridge registration on it.	Dockge and lazydocker are lighter and compose-focused; docker compose itself already does the part that matters, which is why this is the most optional component in the inventory.

service data resolves under one configurable root, so the same file serves both a fresh install and a migration onto existing data, while the memory service's database and cache use named volumes. Two images are built from source and their build contexts are supplied; the retrieval service runs its published upstream image, with the local-build variant given as a commented alternative.

The stack was validated against the live installation rather than merely parsed. The compose configuration resolves cleanly, a dry run produces the complete creation plan, and that plan matches the running containers one for one — identical container names and identical published host ports for all sixteen components.

A.6 Operating envelope: hardware, models and the offline posture

Each component of the plane is an independent application with an interface of its own, and every one of them is open source. Nothing in the design requires them to be used together: the task ledger, the note store, the bibliography manager, the archive reader and the metasearch instance can each be driven directly through their own interface, and the workflow is a convention over them rather than a replacement for them.

The design is aimed at local deployment, and in practice it runs offline. Retrieval, storage, memory, task management and document handling all resolve to services on the local network, and the one component the stack does not containerise — the inference server — is expected on the same network. What the design costs is therefore measured in hardware rather than in subscriptions: system memory, accelerator memory and storage are the binding constraints, and they scale with the number of services enabled and the size of the knowledge base rather than with the number of requests.

Online services remain available at every layer. Storage and compute can be moved off the local machine, and several of the provider-side APIs the workflow reaches — the bibliographic and citation services, the metasearch fallback, the inference provider — are online by construction. Doing so trades two things for convenience: the privacy of the research, because queries, documents and intermediate results leave the machine, and the token cost, because every model call is billed. The local configuration exists to make that trade a deliberate choice rather than a default.

A workable envelope for the whole plane fits on one workstation:

Item	Suggested minimum
System memory	64 GB — the knowledge stores, the graph and vector stores and the memory database are all resident
Accelerator memory	32 GB — holds the main model at quantised weights together with a working context; this applies to the machine that serves the models, which the stack expects on the LAN
Storage	scales with the knowledge base and its embeddings, the container images and the model weights
Main model	QWEN3.8-27B at Q4 quantisation — carries the workflow's main tasks
Memory model	QWEN3-8B — sufficient for the derivation work the memory service does
Embedding model	text-embedding-nomic-embed-text-v1.5 — shared by the memory service and the graph-RAG service

B The Model Context Protocol Bridge

Section 6 described the bridge between the profiles and their tools; this appendix enumerates what crosses it. Every capability the profiles reach without a shell goes through a Model Context Protocol server, and every server is registered in the profile's configuration under a name that becomes the first half of the tool name the model sees. Twelve servers are registered in each profile, exposing seventy-five tools in total, and they divide into two families by construction rather than by purpose.

B.1 Conventions of the bridge

Three conventions govern the whole layer, and each of them is visible in the tables below.

- **Naming.** A tool is addressed as `mcp_` followed by the server name and the tool name, each pair separated by a double underscore — for example the task-creation tool of the ledger server. The name is derived from the registration, not chosen by the server, so renaming a server renames every tool it exposes.
- **Families.** A *native* server speaks the protocol directly: it implements the tool list and the call handler itself. A *CLI-adapter* server is a thin wrapper around an ordinary command-line tool — the adapter introspects the tool's argument parser at startup, publishes each subcommand as a tool with a JSON schema for its arguments, and rebuilds a command line from the call it receives. Nine of the twelve servers are adapters, and the pattern is why adding a new service-backed capability costs a script and a configuration entry rather than protocol code.
- **Credentials.** Every server receives its endpoint and secret through environment variables declared in the registration, not in the script. The scripts are therefore portable between profiles and machines, and the same registration block appears verbatim in both profiles.

The servers divide into those that compute, those that verify and those that retrieve — Tables 14 and 15 — and the two tables together account for the whole registration list of either profile.

Table 14: The computing, verifying and retrieving servers.

Server	Family	Tools	Backing service	What it provides
sagemath	native	10	SageMath, local	Exact symbolic and algebraic computation: solving, factoring, simplification, matrices, number fields, and arbitrary scripts
lean4	native	4	Lean 4 with a pinned Mathlib project	Formal verification: check a file, search the library, run ad-hoc code, attempt an automated proof
wolfram Alpha	CLI adapter	3	Wolfram Alpha SaaS API	Queries whose answer is a number or a closed form rather than a derivation
scientific-coding	CLI adapter	4	Sandboxed Python runtime	Deterministic experiments under CPU and memory caps, with an option to publish results to the note store
searxng	native	1	SearXNG container (Appendix A)	Live federated web search returning structured results
zimi	CLI adapter	4	ZIMI container over ZIM archives	Offline encyclopedia lookups for archival grounding before any live search
lightrag-query	CLI adapter	3	LightRAG container	Multi-hop entity and relation queries over the ingested literature lake

B.2 What each server exposes

Each server is described below in the order the tables introduce it: what it exposes, and how the workflow actually spends it.

sagemath Ten tools over a local SageMath installation: `sage_solve`, `sage_diff`, `sage_integrate`, `sage_simplify`, `sage_factor`, `sage_expand`, `sage_latex`, `sage_matrix`, `sage_number_field` and `sage_run_script`. This is the first stop of the computation stage, and the one that carries exactness: a computation that can be done symbolically is done here rather than numerically, and the `sage_latex` tool exists so that a verified expression can be moved into a manuscript without retyping it.

Table 15: The knowledge, ledger and production servers.

Server	Family	Tools	Backing service	What it provides
vikunja	CLI adapter	15	Vikunja container	The task ledger: projects and tasks, with statuses, due dates and comments; the substrate of hypothesis trees
siyuan	CLI adapter	7	SiYuan container	Block-level notes: create, read, search and update documents and blocks, including equations
zotero	CLI adapter	7	Zotero API	Bibliography: search the library, add by DOI, export BibTeX and synchronise a bibliography file
latex-manuscript	CLI adapter	5	Local TeX Live	Manuscript production: compile, AST-aware edit checks, glossary, notation audit and diff-patch
calibre	native	12	calibre-server on the host	Library management and format conversion; registered in the librarian, environment-only in math

lean4 Four tools against a pinned library project: a file checker, a library search, an ad-hoc evaluation entry point and an automated prover. The verification stage runs on this server alone, and its discipline is what makes the stage worth having — a check must name its project directory explicitly, because a check without a library context compiles against nothing and proves nothing.

wolfram-alpha Three tools wrapping the public API: a short answer, a validation call and a verification call. It is used for questions whose answer is a closed form or a number, in particular when the local stack is unavailable or when an independent implementation is wanted as a cross-check on a SageMath result.

scientific-coding Four tools running Python inside a sandbox with a CPU-second and memory ceiling declared in the registration. This is where numerical experiments, plots and simulations run when they must not touch the host, and one of the tools writes a result into the note store, which is how an experiment's output reaches the knowledge base without a shell command.

searxng A single search tool over the federated metasearch container. It is the live-web arm of the retrieval cascade and is deliberately last in preference order: the cascade tries local sources first and reaches the open web only when they cannot answer.

zimi Four tools over the offline archive container: search, retrieval, reading a document and suggesting a follow-up. It is the archival-grounding step — encyclopedic and historical facts are settled here, in under a second, before a rate-limited web search is spent on them.

lightrag-query Three tools over the graph-RAG container: a query, a streaming query and a data query. It answers multi-hop questions across the ingested literature lake and is the only way the profiles reach the entity graph, which makes the ingestion pipeline a prerequisite for the literature stage rather than a parallel activity.

vikunja Fifteen tools, the largest surface in the bridge, split between projects (list, search, get, create, update) and tasks (list, overdue, search, get, create, update, delete, complete, log). It carries the first and last stages of the workflow: the decomposition stage opens a task and a hypothesis tree through it, and the close-out stage records the outcome in the same structure. Its second role is continuity — the ledger, not the conversation, is what a later session reads to find out what was decided and what is still open.

siyuan Seven tools over the note store: create, read, update and search documents and blocks. It is the human-readable layer of the knowledge base, used when a finding should be legible rather than machine-queryable, and its equation blocks are how a verified formula survives from the computation stage into a manuscript draft.

zotero Seven tools over the bibliography: searching the library, adding an entry by identifier, listing collections, exporting BibTeX and synchronising a bibliography file. It sits immediately after the citation

verification gate in the literature stage — a reference is verified against primary metadata first and only then added to the library, so the library never accumulates a citation that has not been checked.

`latex-manuscript` Five tools over the local TeX installation: compile, an abstract-syntax-aware edit check, glossary generation, a notation audit and a diff-patch applier. This is the manuscript stage's whole tool surface, and its two unusual members are the ones that matter: the edit check catches structural damage that a successful compile would not, and the notation audit catches the same symbol carrying two meanings across sections.

`calibre` Twelve tools for library management and format conversion, registered as a native server in the librarian and as an environment-only block in math — the same name with different meanings in the two profiles, noted in Section 9. Its role is ingestion: converting an acquired book or paper into a form the knowledge base can absorb.

Table 16: Where the bridge enters the six-stage workflow of Section 8.

Stage	Servers	How the stage uses them
1. Problem decomposition	vikunja	The restated problem becomes a task with a hypothesis tree; the tree is what later stages attach results to
2. Literature synthesis	zimi, searxng, lightrag-query, zotero	Archival grounding first, then live search, then multi-hop queries over the ingested lake; a verified reference is added to the bibliography
3. Computation	sagemath, wolfram-alpha, scientific-coding	Exact symbolic work first, an independent implementation as cross-check, sandboxed numerics where the computation must be approximate
4. Formal verification	lean4	Formalise the claim, compile it against a pinned library project, then diff the verified statement against the informal one
5. Manuscript production	latex-manuscript, siyuan	Compile and structurally check the source; the note store carries the notation decisions and equation blocks the draft reuses
6. Reflection and close-out	vikunja, siyuan	Close the tasks, write the outcome into the ledger and the notes, and record the failures that should become skill pitfalls

B.3 Notes on the bridge

The bridge is not capability-gated. Both profiles register the same twelve servers, and all of them are enabled in both. What differs between the profiles is which *skills* are active, not which tools exist: the librarian's skill list disables the computational and proof surface, yet the librarian's configuration still carries a live handle to it. Capability gating at the skill layer and capability gating at the bridge layer are two different things, and only the first is implemented today.

Seventy-five tools are exposed. The count is dominated by the ledger (fifteen) and the library server (twelve); the two servers that carry the intellectual weight of the workflow, the computer-algebra system and the proof assistant, expose ten and four. Tool count is therefore a poor proxy for importance here — a point the abstraction layer of Section 10 has to respect, since a capability gate keyed on the bridge would grant far more than the workflow needs.

One capability of the workflow is not on the bridge at all. Cross-session memory is served by a built-in provider rather than an MCP server, so it is absent from the inventory above by construction: the memory service of Appendix A is reached by the runtime, not by a tool call. The same is true of web search and file access, which are built-in tools rather than registered servers.

Every path in the bridge resolves into the math profile. The registrations of both profiles — including the librarian's, which has no script directory of its own — point at scripts inside the math profile's tree. This is the cross-profile coupling of Section 9 seen from the bridge's side: the bridge is the single largest thing that would break if the math profile were moved or deleted.

C The Skill Surface

Section 5 described the skill catalog as a design; this appendix enumerates it. A skill is a directory holding a `SKILL.md` — frontmatter that states when the skill applies, then an imperative body — together with any scripts, reference notes and credential files it needs. Skills are grouped into category directories, and both profiles carry the same catalog: the same categories, the same entries, and the same two profile-exclusive additions noted at the end. What differs between the profiles is only which entries are switched on.

The first table below is the surface the designated workflow actually uses, stage by stage; the tables after it are the complete catalog, grouped as the loader sees it, with the activation state of each entry in the two profiles.

C.1 The designated workflow surface

The workflow of Section 8 names the skills it relies on, and the table below reproduces that relationship: for each stage, the skills it invokes and what they contribute.

C.2 Notes on the catalog

The two catalogs are the same catalog. Each profile carries fifty-five and fifty-four skill entries respectively — counting the general catalog documented here, not the project-specific skills a particular installation may also hold — and their union is fifty-seven: only five entries are exclusive to one profile — the stack-reconstruction, exact-gate-scripts and profile-packaging skills in math, and the two ingestion skills in the librarian. Everything else is present in both trees, and the profiles differ only in what is switched on.

Activation is a negative specification. Math disables twelve names and the librarian twenty-five, and the librarian's longer list is precisely the computational and proof surface the math profile exists to provide. Resolving those names against the live trees gives three outcomes: five of math's and sixteen of the librarian's disabled names match a skill in the profile tree; six and eight respectively match only an entry in the curator's archive, meaning they are residue from skills already retired; and one name in each list — a local-inference skill that was never installed — matches nothing at all. The distinction matters because resolution is by leaf name: a disabled entry that matches only an archived skill, or nothing, will silently disable a same-named skill the day one is installed.

One gate is inverted. The community-reading skill is disabled for math and enabled for the librarian — the only entry whose activation runs opposite to every other difference between the two profiles, and the clearest evidence that the disabled lists were assembled by hand rather than derived from a role.

Sources and Verification

Every statement in this document is derived from a live installation of the two profiles rather than from recollection: the configuration files, environment files, profile metadata, scheduled-job definitions, script trees, plugin trees, sandbox homes and complete skill trees of both profiles were read directly, together with the shared Hermes plugin pool and the shared MCP interpreter environment. The extraction is reproducible — the script that produced the comparison data and the skill inventory ships in this project, and the same script rebuilds the two profile packs of Section 4.

Table 17: The skills the designated workflow invokes, by stage.

Stage	Skills	What they contribute
1. Problem decomposition	vikunja, mathematical-research	The ledger skill opens the task and the hypothesis tree; the research skill carries the discipline of framing a conjecture before attacking it
2. Literature synthesis	lightrag, academic-research-hub, arxiv, semantic-scholar, searxng, zimi, grounded-citations, literature-project-mapping	Retrieval runs local-first — the graph-RAG lake, then bibliographic search, then the open web and the offline archive — and every recorded claim is grounded in a cited source; the mapping skill links what is found to the projects that need it
2. Literature synthesis (ingestion)	llm-wiki, wiki-pipeline, wiki-maintenance	The intake side: documents are queued, ingested into the graph and the wiki, and the wiki is periodically linted and repaired
3. Computation	sympy-mcp, sagemath-mcp, exact- symbolic-gate-scripts, wolfram-alpha, scientific-coding	Exact symbolic work first, deterministic gate scripts for anything that must be re-runnable, an independent implementation as cross-check, and a resource-capped sandbox for numerics
4. Formal verification	lean4, lean4-goedel-agent	The verification pipeline: formalise an informal statement, then check it against a pinned library project and diff the result against the informal claim
5. Manuscript production	latex-manuscript, user-latex-style, math-article-revision, project-bib, research-paper-writing	Compilation and structural checks, the house style, the revision workflow, bibliography management, and the writing conventions of the field
6. Reflection and close-out	workspace-doc-hierarchy, hermes-agent-skill-authoring, agent-tool-diagnostics	Status is synchronised across the project's documentation, a recurring failure becomes a skill rule rather than a session note, and a tool that looks available but is not is diagnosed rather than worked around

Table 18: Research skills: retrieval, the knowledge base and the workflow itself.

Skill	Role in the workflow	Activation
math-research-workflow	The designated six-stage pipeline for a math project: decomposition, literature, computation, verification, manuscript, reflection, each gated	both profiles
academic-research-hub	Search and download across academic sources; the general entry point when a specific identifier is not known	both profiles
arxiv	Direct arXiv search by keyword, author, category or identifier	both profiles
lightrag	Queries over the ingested literature lake through the graph-RAG service	both profiles
searxng	Live federated web search through the local metasearch instance	both profiles
zimi	Offline encyclopedia and archive lookups for grounding before any live search	both profiles
grounded-citations	The rule that every recorded claim carries a verifiable source, and the procedure for recording it	both profiles
literature-project-mapping	Links a paper found in the wiki to the projects that should act on it	both profiles
llm-wiki	Builds and queries the interlinked markdown knowledge base	both profiles
wiki-pipeline	Document intake through the queue: capture, ingest, index	both profiles
wiki-maintenance	Periodic linting and repair of the knowledge base: orphans, broken links, stale entries	both profiles
rss-feeds	Feed reading and discovery, for monitoring sources that publish on a schedule	disabled in both

Table 19: Research skills: computation, verification and research craft.

Skill	Role in the workflow	Activation
sympy-mcp	Exact symbolic mathematics: solving equations, derivatives, integrals, simplification, matrices	math only
sagemath-mcp	Advanced algebraic structures, arbitrary-precision arithmetic, number fields, and arbitrary scripts	math only
exact-symbolic-gate-scripts	The discipline of writing deterministic, re-runnable exact gate scripts, so a claim can be re-checked without a language model	math only
wolfram-alpha	Concise computational answers and an independent implementation to cross-check a local result	both profiles
scientific-coding	Python experiments, simulations and plots inside a CPU- and memory-capped sandbox	math only
lean4	Checking Lean files, searching the library, inspecting goal states, attempting a proof	math only
lean4-goedel-agent	The formalisation pipeline: informal statement, formal statement, then a fidelity diff between the two	math only
mathematical-research	The conduct of research itself: forming conjectures, reviewing the literature, attempting proofs	both profiles
theoretical-research-phase	Phases that need no computation: framing, planning and conceptual work	both profiles
math-article-revision	The multi-edit revision workflow for a mathematical manuscript	both profiles
research-paper-writing	Writing conventions and the submission workflow for conference papers	both profiles
agent-tool-diagnostics	Diagnosing a tool that appears available in the runtime but does not work	both profiles
hermes-tool-edge-cases	Known tool quirks and their workarounds: editing LaTeX, reading long files, terminal transport of long commands	both profiles

Table 20: Productivity and operations skills: the knowledge stores, the manuscript toolchain and the local stack.

Skill	Role in the workflow	Activation
vikunja	Task ledger operations: projects, tasks, statuses, due dates and comments — the workflow's continuity layer	both profiles
siyuan	Block-level notes: creating, reading, searching and updating documents and equation blocks	both profiles
zotero	Bibliography: searching the library, adding entries by identifier, exporting BibTeX	both profiles
latex-manuscript	Manuscript production: compilation, structure-aware edit checks, glossary, notation audit, diff-patch	both profiles
ocr-and-documents	Text extraction from PDFs and scans, including the OCR path	both profiles
project-bib	Per-project bibliography management and synchronisation	math only
session-librarian	Organising the session record: locating, renaming, archiving and pruning sessions	math only
calibre	Ebook library management and format conversion	disabled in both
calibre-library-ingestion	Ingesting an existing library into the knowledge base	librarian only
djvu-to-pdf	Format conversion for scanned books	librarian only
pocketbase	API operations on the intake queue that feeds the wiki pipeline	both profiles
lmstudio-configuration	Troubleshooting the local inference server: duplicate models, paths, service state	both profiles
llama-cpp	Local inference through a second runtime, kept as a fallback	disabled in both

Table 21: Engineering, automation and access skills.

Skill	Role in the workflow	Activation
hermes-agent	Operating and configuring the agent runtime itself: profiles, skills, plugins, schedules	both profiles
hermes-agent-skill-authoring	Writing a new skill to the house format, so a lesson becomes reusable procedure	both profiles
hermes-desktop-troubleshooting	Diagnosing the desktop application: panes, updates, rendering	both profiles
plan	Writing a plan to disk as an artifact, without executing it	both profiles
subagent-driven-development	Executing a plan through delegated workers with a review gate on each stage	both profiles
dynamic-workflow	Orchestrating large fan-out work as a plan expressed in code	both profiles
inspecting-hermes-desktop	Reading the live desktop interface through the debugging protocol	math only
sdic-review	Reviewing handoffs and routing verified outcomes	math only
docker-stack-reconstruction	Rebuilding and validating a containerised service stack from its live state	math only
hermes-profile-packaging	Producing a sanitised, shareable copy of a profile: exclusion and scrub rules	math only
github	Repository operations through the command line: pull requests, issues, reviews	math only
github-repo-management	Cloning, creating and forking repositories; remotes and releases	math only
merge-reconciler	Reconciling divergent branches after a merge conflict	disabled in both
github-auth	Setting up and repairing repository authentication	disabled in both
blocked-page-recovery	Reaching content behind access walls, rate limits and bot filters	both profiles
reddit-reading	Community reading; the only skill whose gate is inverted between the profiles	librarian only

Table 22: Top-level skills: capabilities that cut across the categories.

Skill	Role in the workflow	Activation
user-latex-style	The house LaTeX style: document class, fonts, notation and the anti-pattern checklist every manuscript passes	both profiles
semantic-scholar	The citation-graph API: search, references, recommendations, batch enrichment	both profiles
hermes-plugin-management	Installing, enabling and debugging plugins	both profiles
workspace-doc-hierarchy	Maintaining the layered documentation chain: project guidance at each level	math only

- **Extraction** — a script in `references/` reads both profiles and writes the comparison data and the per-entry skill inventory (`skill_inventory.md`).
- **Packs** — the pack builder in `references/` produces the `math/` and `librarian/` folders from the same sources (`build_profile_packs.py`).
- **Structural claims** — the cross-profile path coupling and the curator tombstones were each confirmed by direct filesystem and configuration inspection, not inferred.

Redaction note: no credential, endpoint, hostname or personal identifier appears anywhere in this document or in the shipped packs. Secrets are replaced by placeholders in the packs and masked in the extracted data artifacts; the redaction pass is applied mechanically by the pack builder and verified by a residual-secret scan over the result.